

# load best practices

## load best practices

数据导入是 Apache Doris 最核心的能力之一，导入效率直接决定了实时分析的时效性与系统稳定性。本文从典型业务场景出发，先给出可直接落地的最佳实践，再深入剖析导入原理与调优思路，帮助你：

- 根据业务特征选择合适的导入方式
- 在表设计、攒批、分桶、内存、并发等环节做出合理决策
- 规避高频小导入、写放大、Memtable OOM 等常见问题
- 在延迟与吞吐之间找到与业务匹配的平衡点

### 快速导航

- 关注点：选择导入方式；推荐做法：按数据源与实时性选型；详情章节：选择合适的导入方式
- 关注点：表模型；推荐做法：优先使用明细模型；详情章节：表模型与索引
- 关注点：分区分桶；推荐做法：单 Tablet 控制在 1-10 GB；详情章节：分区分桶配置
- 关注点：Random 分桶；推荐做法：开启 load\_to\_single\_tablet 提升吞吐；详情章节：Random 分桶导入优化
- 关注点：高频小导入；推荐做法：客户端攒批 + Group Commit；详情章节：攒批策略
- 关注点：单次导入分区数；推荐做法：控制少量分区，避免 Memtable 过多；详情章节：分区导入与内存控制
- 关注点：大数据量导入；推荐做法：分批导入，单批  $\leq 100$  GB；详情章节：大规模数据分批导入
- 关注点：Broker Load 并发；推荐做法：拆分压缩 / 列存文件以并发；详情章节：Broker Load 并发
- 关注点：Stream Load 并发；推荐做法：单 BE 并发不超过 128；详情章节：Stream Load 并发

## 选择合适的导入方式

Doris 提供多种导入方式，共享相同的核心流程，但针对不同场景做了优化。可根据数据源和业务需求选择：

- 导入方式：**Stream Load**；接入协议：HTTP；执行模式：同步；典型数据源：本地文件、数据流；适用场景：实时写入（如应用程序推送数据）
- 导入方式：**Broker Load**；接入协议：SQL；执行模式：异步；典型数据源：HDFS、S3 等外部存储；适用场景：大规模批量导入
- 导入方式：**Routine Load**；接入协议：SQL；执行模式：异步流式；典型数据源：Kafka；适用场景：实时同步消息队列数据，支持 Exactly-Once
- 导入方式：**Insert Into / Select**；接入协议：SQL；执行模式：同步/异步；典型数据源：Doris 表、Hive、MySQL、S3 TVF；适用场景：ETL 作业、外部数据集成
- 导入方式：**MySQL Load**；接入协议：MySQL LOAD DATA；执行模式：同步；典型数据源：本地 CSV 文件；适用场景：小规模测试或 MySQL 用户迁移（FE 转发为 Stream Load）

## 导入最佳实践

下面从用户在数据导入中关心的几个典型问题展开，给出可直接落地的最佳实践。

### 表模型与索引

#### 表模型选择

建议优先考虑使用 **明细模型（Duplicate Key）**：

- 仅存储原始数据，无需聚合或去重计算，导入路径最短；
- 在数据导入和查询性能方面相比其他模型都具有优势；
- Aggregate 模型需按 Key 列聚合、Unique Key 模型需去重，均会增加 BE 在 MemTable 阶段的额外计算（排序、去重），增加 CPU 与内存消耗。

仅在确实需要按 Key 聚合或去重时再选择 Aggregate 或 Unique Key 模型。如需了解更多信息，请参考：数据模型。

#### 索引控制

索引（如位图索引、倒排索引）需要在导入时同步更新，会增加写入维护成本。建议：

- 仅为高频查询字段创建索引；

- 避免冗余索引，减少 BE 写入时的索引构建与校验，从而降低 CPU 和内存占用，提升导入吞吐量。

## 分区分桶配置

分区与分桶决定了数据在 BE 节点上的分布与并行度，直接影响导入与查询性能。

### 单 Tablet 大小：1-10 GB

建议将单个 Tablet 的大小控制在 **1-10 GB** 范围内：

- Tablet 大小：过小；影响：聚合效果不佳，元数据管理压力增加
- Tablet 大小：过大；影响：不利于副本迁移、补齐
- Tablet 大小：**1-10 GB（推荐）**；影响：兼顾导入、查询与副本管理

分桶数计算公式：分桶数 = 总数据量 / (1-10 GB)，并配合合适的分桶键（如随机数列）避免数据倾斜，平衡 BE 节点负载，提升并行写入效率。

详细参考：数据分布。

### 分区设计：减少分发与内存压力

按业务查询模式（如时间、区域）划分分区，导入时数据仅分发至目标分区，避免处理无关分区的元数据与文件。同时：

- 同时写入多个分区会激活大量 Tablet，每个 Tablet 占用独立的 MemTable，显著增加 BE 内存压力；
- 内存紧张时会触发提前 Flush，生成大量小 Segment 文件，引发频繁 Compaction 与写放大。

通过限制活跃分区数量（如逐天导入），可减少同时活跃的 Tablet 数，缓解内存紧张，生成更大的 Segment 文件，降低 Compaction 负担，从而提升并行写入效率和后续查询性能。

## Random 分桶导入优化

在使用 Random 分桶时，可以通过设置 `load_to_single_tablet=true` 启用单分片导入模式：

- 数据直接写入单一 Tablet，绕过分发到多个 Tablet 的过程，消除 Tablet 分布计算的 CPU 开销与 BE 间的 RPC 传输开销；
- 集中写入单一 Tablet 减少了小 Segment 文件的生成，避免频繁 Compaction 带来的写放大；

- 显著提升大规模数据导入的并发度与吞吐量。

详细参考：Random 分桶。

### 攒批策略

每次导入都是一个独立事务，涉及 FE 的 Edit Log 写入（记录元数据变更）和 BE 的 MemTable 刷盘（生成 Segment 文件）。**高频小批量导入（KB 级别）** 会带来：

- FE Edit Log 频繁写入，增加 FE 磁盘 I/O；
- BE MemTable 频繁刷盘，生成大量小 Segment 文件；
- 触发频繁 Compaction，引发严重写放大。

可结合客户端与服务端两种攒批方式来缓解：

- 攒批方式：**客户端攒批**；适用场景：客户端可控的批量写入；推荐做法：在客户端将数据攒到 **数百 MB 到数 GB** 后再发起导入。单次大事务替代多次小事务，可降低 FE 的 Edit Log 写入频率及 BE 的 MemTable 刷盘次数，避免存储碎片和后续 Compaction 的资源消耗
- 攒批方式：**服务端攒批（Group Commit）**；适用场景：高并发、小数据量导入；推荐做法：开启 Group Commit，由服务端将短时间内多个小批量合并为单一事务。合并后的大事务生成更大的 Segment 文件，减轻后台 Compaction 压力，特别适用于日志、IoT 数据等高频小批量场景

### 分区导入与内存控制

数据导入时，BE 先将数据写入内存的 MemTable（默认 200 MB），写满后异步刷盘生成 Segment 文件。每个 Tablet 在内存中存在一个活跃的 MemTable，因此单次导入覆盖的分区数会直接影响内存占用：

1. 分区过多 → 活跃 Tablet 多 → 活跃 MemTable 多 → 内存占用升高；
2. 内存占用过高时，为避免进程 OOM 会被提前下刷；
3. 提前下刷会产生大量小文件，进一步影响导入性能。

为获得稳定的导入吞吐：

- **按分区顺序导入**：例如逐天导入，集中数据写入单一分区，减少 MemTable 分散和刷盘次数，降低内存碎片和 I/O 压力；
- **严格控制单次导入涉及的分区数量**，避免因 MemTable 过多触发提前刷盘。

## 大规模数据分批导入

在文件数较多、数据量很大时，建议**分批进行导入**，以降低重试代价并减少对 BE 内存与磁盘的集中冲击：

- 数据来源：HDFS / 对象存储等远端文件；推荐导入方式：Broker Load；单批数据量建议：单批 **不超过 100 GB**
- 数据来源：本地大数据量文件；推荐导入方式：Doris streamloader 工具（自动分批）；单批数据量建议：由工具自动控制

## Broker Load 并发

不同文件类型对应的并发策略有所不同：

- **压缩文件 / Parquet / ORC 文件**：建议将文件分割为多个小文件后导入，以实现多并发导入；
- **非压缩 CSV 与 JSON 文件**：Doris 内部会自动切分文件并并发导入。

并发数策略请参考：Broker Load 导入配置参数。

## Stream Load 并发

Stream Load 的并发受 BE 端线程池参数限制，建议遵循以下阈值：

- 参数 / 阈值：单 BE 并发数（推荐上限）；推荐值：**不超过 128**；说明：由 BE 的 webserver\_num\_workers 参数控制；超出可能导致 webserver 线程不够用，影响导入性能
- 参数 / 阈值：单 BE 并发数（强限制）；推荐值：**不应超过 512**；说明：由 doris\_max\_remote\_scanner\_thread\_pool\_thread\_num 参数控制；超过该值可能导致 BE 进程卡住

## 低时延场景的并发取舍

对低时延要求场景（如实时监控）：

- 适当 **降低并发数**，避免资源争抢；
- 结合 Group Commit 的异步模式（async\_mode）合并小事务，减少事务提交延迟。

## 深入了解 Doris 导入原理

掌握以下原理有助于在复杂场景下做出更合理的调优决策。

## 设计理念

Doris 的数据导入建立在其分布式架构之上，主要涉及前端节点（Frontend, FE）和后端节点（Backend, BE）：

- **FE**：负责元数据管理、查询解析、任务调度和事务协调；
- **BE**：负责实际的数据存储、计算与写入操作。

数据导入设计旨在满足实时写入、流式同步、批量加载和外部数据源集成等多样化业务需求，其核心理念包括：

- 设计目标：**一致性与原子性**；实现方式：每个导入任务作为一个事务，确保数据原子写入，避免部分写入。通过 Label 机制保证导入数据的不丢不重。
- 设计目标：**灵活性**；实现方式：支持多种数据源（如本地文件、HDFS、S3、Kafka 等）和格式（如 CSV、JSON、Parquet、ORC 等），满足不同场景。
- 设计目标：**高效性**；实现方式：利用分布式架构并行处理数据，多 BE 节点并行处理数据，提高吞吐量。
- 设计目标：**简易性**；实现方式：提供轻量级 ETL 功能，用户可在导入时直接进行数据清洗和转换，减少外部工具依赖。
- 设计目标：**灵活建模**；实现方式：支持明细模型（Duplicate Key）、主键模型（Unique Key）和聚合模型（Aggregate Key），允许在导入时进行数据聚合或去重。

## 通用导入流程

无论使用何种导入方式（Stream Load、Broker Load、Routine Load 等），核心流程基本一致：

### 1. 提交导入任务

1. 用户通过客户端（如 HTTP、JDBC、MySQL 客户端）提交导入请求，指定数据源（如本地文件、Kafka Topic、HDFS 文件路径）、目标表、文件格式和导入参数（如分隔符、错误容忍度）。
2. 每个任务可以指定一个唯一的 **Label**，用于标识任务并支持幂等性（防止重复导入）。例如，用户在 Stream Load 中通过 HTTP header 指定 Label。
3. Doris 的前端节点（FE）接收请求，验证权限、检查目标表是否存在，并解析导入参数。

### 2. 任务分配与协调

1. FE 分析数据分布（基于表的分区和分桶规则），生成导入计划，并选择一个后端节点（BE）作为 **Coordinator**，负责协调整个任务。
2. 如果用户直接向 BE 提交（如 Stream Load），BE 可直接担任 Coordinator，但仍需从 FE 获取元数据（如表 Schema）。
3. 导入计划会将数据分配到多个 BE 节点，确保并行处理以提高效率。

### 3. 数据读取与分发

1. Coordinator BE 从数据源读取数据（例如，从 Kafka 拉取消息、从 S3 读取文件，或直接接收 HTTP 数据流）。
2. Doris 解析数据格式（如对 CSV 分割、JSON 解析），并支持用户定义的 **轻量 ETL** 操作，包括：
  - **前置过滤**：对原始数据进行过滤，减少处理开销；
  - **列映射**：调整数据列与目标表列的对应关系；
  - **数据转换**：通过表达式处理数据；
  - **后置过滤**：对转换后的数据进行过滤。
3. Coordinator BE 解析完数据后按分区和分桶规则分发到多个下游的 Executor BE。

### 4. 数据写入

1. 数据分发到多个 BE 节点，写入内存表（MemTable），按 Key 列进行排序。对于 Aggregate 或 Unique Key 模型，Doris 会根据 Key 进行聚合或去重（如 SUM、REPLACE）。
2. 当 MemTable 写满（默认 200 MB）或任务结束时，数据异步写入磁盘，形成列式存储的 **Segment 文件**，并组成 **Rowset**。
3. 每个 BE 独立处理分配的数据，写入完成后向 Coordinator 报告状态。

### 5. 事务提交与发布

1. Coordinator 向 FE 发起事务提交（Commit）。FE 确保多数副本成功写入后，通知 BE 发布数据版本（Publish Version），等 BE Publish 成功后，FE 标记事务为 **VISIBLE**，此时数据可查询。
2. 如果失败，FE 触发回滚（Rollback），删除临时数据，确保数据一致性。

### 6. 结果返回

1. 同步方式（如 Stream Load、Insert Into）直接返回导入结果，包含成功/失败状态和错误详情（如 ErrorURL）。
2. 异步方式（如 Broker Load）提供任务 ID 和 Label，用户可通过 SHOW LOAD 查看进度、错误行数和详细信息。

3. 操作记录到审计日志，支持后续追溯。

## MemTable 前移

MemTable 前移是 Apache Doris 2.1.0 版本引入的优化机制，针对 INSERT INTO... SELECT 导入方式显著提升性能：

- **传统流程瓶颈：**Sink 节点需将数据编码为 Block 格式，通过 Ping-pong RPC 传输到下游节点，涉及多次编码和解码，增加开销；
- **优化方式：**Sink 节点直接处理 MemTable，生成 Segment 数据后通过 Streaming RPC 传输，减少编码解码和传输等待，同时提供更准确的内存反压；
- **性能提升：**官方测试显示在单副本场景下导入耗时降低至 36%，三副本场景下降低至 54%，整体性能提升超 100%；
- **适用范围：**目前该功能只支持存算一体部署模式。

## 存算分离架构下的导入

在存算分离架构下，导入优化聚焦于数据存储和事务管理的解耦：

- **数据存储**
  - BE 不持久化数据，MemTable Flush 后生成 Segment 文件直接上传至共享存储（如 S3、HDFS），利用对象存储的高可用性和低成本支持弹性扩展；
  - BE 本地 File Cache 异步缓存热点数据，通过 TTL 和 Warmup 策略提升查询命中率；
  - 元数据（如 Tablet、Rowset 元数据）由 Meta Service 存储于 FoundationDB，而非 BE 本地 RocksDB。
- **事务处理**
  - 事务管理从 FE 迁移至 Meta Service，消除 FE Edit Log 写入瓶颈；
  - Meta Service 通过标准接口（beginTransaction、commitTransaction）管理事务，依赖 FoundationDB 的全局事务能力确保一致性；
  - BE 协调者直接与 Meta Service 交互，记录事务状态，通过原子操作处理冲突和超时回收，简化同步逻辑，提升高并发小批量导入吞吐量。

## 延迟与吞吐的取舍

数据导入的 **延迟（Latency）** 与 **吞吐量（Throughput）** 往往需要在实际业务场景中进行平衡：

- **更低延迟**：意味着用户能更快看到最新数据，但写入批次更小，写入频率更高，会导致后台 Compaction 更频繁，占用更多 CPU、IO 和内存资源，同时增加元数据管理的压力；
- **更高吞吐**：通过增大单次导入数据量来减少导入次数，可以显著降低元数据压力和后台 Compaction 开销，从而提升系统整体性能。但数据写入到可见之间的延迟会有所增加。

因此，建议用户在满足业务时延要求的前提下，**尽量增大单次导入写入的数据量**，以提升吞吐并减少系统开销。

## 测试数据

### Flink 端到端时延

采用 Flink Connector 使用攒批模式进行写入，主要关注数据端到端的时延和导入吞吐。攒批时间通过 Flink Connector 的 `sink.buffer-flush.interval` 参数来控制，Flink Connector 的详细使用参考 Flink-Doris-Connector。

### 机器配置：

- 1 台 FE：8 核 CPU、16 GB 内存
- 3 台 BE：16 核 CPU、64 GB 内存

### 数据集：

- TPC-H lineitem 数据

不同攒批时间和不同并发下的导入性能，测试结果：

- 攒批时间 (s) : 0.2; 导入并发: 1; bucket 数: 32; 吞吐 (rows/s) : 6073; 端到端平均时延 (s) : 0.211; 端到端 P99 时延 (s) : 0.517
- 攒批时间 (s) : 1; 导入并发: 1; bucket 数: 32; 吞吐 (rows/s) : 31586; 端到端平均时延 (s) : 0.71; 端到端 P99 时延 (s) : 1.39
- 攒批时间 (s) : 10; 导入并发: 1; bucket 数: 32; 吞吐 (rows/s) : 67437; 端到端平均时延 (s) : 5.65; 端到端 P99 时延 (s) : 10.90
- 攒批时间 (s) : 20; 导入并发: 1; bucket 数: 32; 吞吐 (rows/s) : 93769; 端到端平均时延 (s) : 10.962; 端到端 P99 时延 (s) : 20.682
- 攒批时间 (s) : 60; 导入并发: 1; bucket 数: 32; 吞吐 (rows/s) : 125000; 端到端平均时延 (s) : 32.46; 端到端 P99 时延 (s) : 62.17
- 攒批时间 (s) : 0.2; 导入并发: 10; bucket 数: 32; 吞吐 (rows/s) : 9300; 端到端平均时延 (s) : 0.38; 端到端 P99 时延 (s) : 0.704

- 攒批时间 (s) : 1; 导入并发: 10; bucket 数: 32; 吞吐 (rows/s) : 34633; 端到端平均时延 (s) : 0.75; 端到端 P99 时延 (s) : 1.47
- 攒批时间 (s) : 10; 导入并发: 10; bucket 数: 32; 吞吐 (rows/s) : 82023; 端到端平均时延 (s) : 5.44; 端到端 P99 时延 (s) : 10.43
- 攒批时间 (s) : 20; 导入并发: 10; bucket 数: 32; 吞吐 (rows/s) : 139731; 端到端平均时延 (s) : 11.12; 端到端 P99 时延 (s) : 22.68
- 攒批时间 (s) : 60; 导入并发: 10; bucket 数: 32; 吞吐 (rows/s) : 171642; 端到端平均时延 (s) : 32.37; 端到端 P99 时延 (s) : 61.93

不同 bucket 数对导入性能的影响, 测试结果:

- 攒批时间 (s) : 1; 导入并发: 10; bucket 数: 4; 吞吐 (rows/s) : 34722; 端到端平均时延 (s) : 0.86; 端到端 P99 时延 (s) : 2.28
- 攒批时间 (s) : 1; 导入并发: 10; bucket 数: 16; 吞吐 (rows/s) : 34526; 端到端平均时延 (s) : 0.8; 端到端 P99 时延 (s) : 1.52
- 攒批时间 (s) : 1; 导入并发: 10; bucket 数: 32; 吞吐 (rows/s) : 34633; 端到端平均时延 (s) : 0.75; 端到端 P99 时延 (s) : 1.47
- 攒批时间 (s) : 1; 导入并发: 10; bucket 数: 64; 吞吐 (rows/s) : 34829; 端到端平均时延 (s) : 0.81; 端到端 P99 时延 (s) : 1.51
- 攒批时间 (s) : 1; 导入并发: 10; bucket 数: 128; 吞吐 (rows/s) : 34722; 端到端平均时延 (s) : 0.83; 端到端 P99 时延 (s) : 1.55

### Group Commit 测试

小批量高频导入建议开启 Group Commit, 可以大幅提升导入性能。Group Commit 性能测试数据参考 Group Commit 性能。

## FAQ

### Q1: 如何选择合适的导入方式?

按数据源和实时性需求选择: 本地文件实时推送用 Stream Load; HDFS/S3 大批量数据用 Broker Load; Kafka 流式数据用 Routine Load; 外表或 Doris 表之间的 ETL 用 Insert Into / Select; MySQL 兼容场景用 MySQL Load。

### Q2: 为什么高频小批量导入会显著影响性能?

高频小导入会导致 Doris 频繁进行 Compaction, 进而引发严重的写放大问题。建议在客户端攒批至数 MB 至数 GB, 或开启 Group Commit 在服务端完成攒批。

### Q3: 导入产生大量小文件、Compaction 压力大怎么办?

优先开启 Group Commit 合并小事务，或在客户端攒批至数百 MB 到数 GB 再导入；同时检查分区与分桶设计，避免活跃 Tablet 过多。

#### **Q4：为什么单次导入不要覆盖过多分区？**

每个 Tablet 在内存中持有一个活跃 Memtable。当活跃 Memtable 总占用内存过高时，会提前下刷以避免 OOM，从而生成大量小文件并降低导入性能。控制单次导入的分区数量可以缓解该问题。

#### **Q5：单次导入数据量是否越大越好？**

不是。建议单批次  $\leq 100$  GB，过大会增加重试代价和 BE 内存/磁盘冲击。本地大文件可借助 streamloader 工具自动分批。

#### **Q6：Tablet 大小为何建议在 1-10 GB？**

过小的 Tablet 会削弱聚合效果并加重元数据管理压力；过大的 Tablet 在副本迁移和补齐时成本更高。1-10 GB 是兼顾导入、查询与副本管理的经验区间。

#### **Q7：Stream Load 并发应如何设置？**

单 BE 上的 Stream Load 并发建议不超过 128（受 webserver\_num\_workers 限制），且必须低于 512（受 doris\_max\_remote\_scanner\_thread\_pool\_thread\_num 限制），否则可能造成 BE 进程卡住。

#### **Q8：MemTable 前移在哪些场景生效？**

仅对存算一体部署模式下的 INSERT INTO...SELECT 生效，单副本耗时降至原来的 36%、三副本降至 54%。存算分离模式当前不支持该优化。

#### **Q9：如何避免数据倾斜？**

合理设置分桶数（单 Tablet 压缩后 1-10 GB）并选择合适的分桶键；随机分桶场景可启用 load\_to\_single\_tablet=true 减少分发开销。

## **Troubleshooting**

- 现象：导入吞吐低、Compaction 频繁；可能原因：高频小批量导入引发写放大；排查与解决：客户端攒批，或开启 Group Commit
- 现象：导入过程中产生大量小文件；可能原因：单次导入覆盖分区过多，Memtable 提前下刷；排查与解决：减少单次导入涉及的分区数；按分区顺序导入

- 现象：导入提交后长时间不可见；可能原因：副本未达多数派、Publish 慢；排查与解决：通过 SHOW LOAD 查看任务状态；确认 BE 健康，必要时调大 publish 超时
- 现象：BE OOM 或频繁 Flush；可能原因：同时活跃分区/Tablet 过多，MemTable 分散；排查与解决：按分区顺序导入；减少分桶数；降低并发
- 现象：FE Edit Log 压力大；可能原因：高频小批量导入，事务过多；排查与解决：开启 Group Commit，或在客户端攒批；评估并发是否过高
- 现象：Compaction 跟不上、查询变慢；可能原因：小文件过多，写放大严重；排查与解决：加大单次导入数据量；优化分桶；必要时调整 Compaction 参数
- 现象：存算分离下导入抖动；可能原因：对象存储 QPS 限流；排查与解决：控制并发；增大单批数据量；与对象存储侧确认 QPS 配额
- 现象：导入报错有错误行；可能原因：数据格式不符、列映射错误；排查与解决：通过返回的 ErrorURL 查看错误样本；调整列映射、分隔符或错误容忍度
- 现象：BE 进程卡住；可能原因：Stream Load 单 BE 并发超过 512；排查与解决：降低并发，调整 `doris_max_remote_scanner_thread_pool_thread_num`
- 现象：webserver 线程不够用、Stream Load 慢；可能原因：单 BE 并发超过 128；排查与解决：控制并发在 128 以内或调整 `webserver_num_workers`
- 现象：Broker Load 并发低；可能原因：单个压缩 / Parquet / ORC 文件过大且未拆分；排查与解决：将大文件拆分为多个小文件后再导入
- 现象：单次 Broker Load 失败重试代价大；可能原因：单批数据量过大；排查与解决：单批控制在 100 GB 以内并分批提交